Service Catalog API

Technical Documentation

Django + Django REST Framework

Version 1.0
August 2026

Table of Contents

1. Overview

This document describes the Service Catalog module: a Django app that models the services and features an organization offers (by audience segment), and exposes them through a versioned REST API. The module is organized in four layers so that business logic stays reusable outside the HTTP layer (admin actions, management commands, background jobs).

- Models — database schema, constraints, and soft-delete/audit behavior
- Services — business logic (creation, updates, reordering, soft delete/restore)
- Serializers — request/response shape and field-level validation
- Views — HTTP endpoints, implemented as DRF APIView subclasses

Note on data classification: these models represent catalog metadata (which services/features are offered) and do not themselves store patient health information (PHI). If this module is later linked to clinical or patient records, those linked models require additional controls (field-level encryption, stricter access control, and request-level audit logging) beyond what is described here.

2. Architecture

Request flow through the module:

```
Client
  |
  v
urls.py            routes HTTP method + path to a view class
  |
  v
views.py           APIView subclasses: auth, permissions, filtering,
                   pagination, request/response wiring
  |
  v
serializers.py     validates & shapes input/output, no DB writes
  |
  v
services.py        business logic, transactions, bulk DB operations
  |
  v
models.py          schema, constraints, managers (soft delete / active)
```

Views never write to the database directly for create/update flows — they validate input via a serializer, then hand the validated data to a function in services.py. This keeps a single source of truth for business rules regardless of how they're invoked.

3. Data Model

3.1 Feature

Represents a single feature that can be attached to one or more services.

Field	Type	Notes
id	AutoField	Primary key
name	CharField(100)	Unique

description	TextField	Optional
is_active	BooleanField	Business toggle; indexed
is_deleted / deleted_at	BooleanField / DateTimeField	Soft delete
created_by / updated_by	FK → User	Nullable, SET_NULL on user deletion
created_at / updated_at	DateTimeField	Auto-managed timestamps

3.2 Service

Represents an offered service, scoped to an audience.

Field	Type	Notes
id	AutoField	Primary key
code	SlugField(50)	Unique; lowercase, hyphenated
name	CharField(100)	Unique together with service_for
service_for	CharField(20, choices)	individual / family / executive / organization
description	TextField	Optional
features	M2M → Feature	Through ServiceFeature (ordered)
is_active	BooleanField	Business toggle; indexed
is_deleted / deleted_at	BooleanField / DateTimeField	Soft delete
created_by / updated_by	FK → User	Nullable, SET_NULL on user deletion
created_at / updated_at	DateTimeField	Auto-managed timestamps

3.3 ServiceFeature (through table)

Links a Service to a Feature with an explicit display order.

Field	Type	Notes
service	FK → Service	CASCADE
feature	FK → Feature	CASCADE
display_order	PositiveSmallIntegerField	Default 0; unique per (service, feature)

3.4 Managers

- Model.objects — default manager; excludes soft-deleted AND inactive rows
- Model.all_objects — unfiltered; used for staff views and admin/audit tooling

4. Business Rules

- is_active vs is_deleted are independent: is_active is a merchandising toggle; is_deleted retires a record while preserving it for audit history.
- Deleting a Service or Feature through the API performs a soft delete, never a hard DELETE.
- A (service, feature) pair is unique — a feature can only be attached to a given service once.
- Reordering features on a service is a full replace of that service's ServiceFeature rows, executed as bulk_create / bulk_update / delete inside a single transaction.
- Service name must be unique within its service_for audience, not globally.

5. API Reference

Base path: /api/catalog/ (mount point is defined by the parent project's URLconf.)

5.1 Features

Method	Path	Auth	Description
GET	/features/	Public	List features (paginated, searchable, orderable)
POST	/features/	Staff	Create a feature
GET	/features/{id}/	Public	Retrieve a feature
PUT / PATCH	/features/{id}/	Staff	Update a feature
DELETE	/features/{id}/	Staff	Soft-delete a feature
POST	/features/{id}/restore/	Staff	Restore a soft-deleted feature

5.2 Services

Method	Path	Auth	Description
GET	/services/	Public	List services (filter by service_for/is_active, search, order)
POST	/services/	Staff	Create a service, optionally with a features list
GET	/services/{id}/	Public	Retrieve a service with its ordered features
PUT / PATCH	/services/{id}/	Staff	Update a service and/or replace its features
DELETE	/services/{id}/	Staff	Soft-delete a service
POST	/services/{id}/restore/	Staff	Restore a soft-deleted service
POST	/services/{id}/reorder-features/	Staff	Set a new display order for attached features

5.3 List query parameters

Parameter	Applies to	Example
search	features, services	?search=annual
ordering	features, services	?ordering=-created_at
service_for	services	?service_for=family
is_active	services	?is_active=true
page / page_size	features, services	?page=2&page_size=50 (max 100)

6. Permissions & Security

- IsStaffOrReadOnly: GET/HEAD/OPTIONS are open to any client, including anonymous; all writes require request.user.is_staff.
- Staff-authenticated requests see soft-deleted and inactive rows (via all_objects); everyone else sees only the live catalog (via objects).
- Serializers whitelist fields explicitly — no fields = "__all__" — so new model fields are never exposed until deliberately added.
- created_by / updated_by are set server-side from request.user and are never accepted from client input.

For finer-grained roles than Django's is_staff flag (e.g. a dedicated catalog-manager group), replace the check in IsStaffOrReadOnly with request.user.has_perm("catalog.change_service").

7. Request / Response Examples

7.1 Create a service with features

```
POST /api/catalog/services/
Authorization: Bearer <staff token>

{
  "code": "annual-checkup",
  "name": "Annual Checkup",
  "service_for": "individual",
  "description": "Yearly preventive checkup.",
  "is_active": true,
  "features": [
    { "feature_id": 3, "display_order": 0 },
    { "feature_id": 7, "display_order": 1 }
  ]
}
```

7.2 Reorder features on a service

```
POST /api/catalog/services/12/reorder-features/
Authorization: Bearer <staff token>
```

```
{ "feature_ids": [7, 3, 9] }
```

7.3 Retrieve a service

```
GET /api/catalog/services/12/

{
  "id": 12,
  "code": "annual-checkup",
  "name": "Annual Checkup",
  "service_for": "individual",
  "description": "Yearly preventive checkup.",
  "features": [
    {
      "id": 7,
      "name": "Lab Panel",
      "description": "...",
      "display_order": 0
    },
    {
      "id": 3,
      "name": "Physician Consult",
      "description": "...",
      "display_order": 1
    }
  ],
  "is_active": true,
  "created_at": "2026-08-01T09:00:00Z",
  "updated_at": "2026-08-20T14:12:00Z"
}
```

8. Deployment & Scalability Notes

- Install dependencies: djangorestframework, django-filter (add django_filters to INSTALLED_APPS).
- Set DEFAULT_PAGINATION_CLASS and DEFAULT_THROTTLE_RATES in DRF settings; write endpoints should have tighter throttle rates than reads.
- List/retrieve queries prefetch service_features__feature to avoid N+1 queries when serializing nested features.
- The catalog is read-heavy and changes infrequently — cache list/retrieve responses (e.g. cache_page or a manual cache-and-invalidate-on-save pattern).
- Apply the model-level indexes and unique constraints via migrations before deploying; check for pre-existing duplicate (name, service_for) pairs first, since the constraint will fail to apply otherwise.
- Consider django-simple-history if full field-level change history is required for compliance audits beyond the created_by/updated_by columns.

9. Appendix: File Map

File	Responsibility
models.py	Schema, constraints, indexes, managers
serializers.py	Input validation, output shaping

services.py	Business logic, transactions, bulk operations
permissions.py	IsStaffOrReadOnly access rule
filters.py	django-filter FilterSet for service queries
views.py	APIView subclasses (HTTP layer)
urls.py	Path-based routing